

Since you're preparing for **Python interviews** and **Data Engineering**, here's a **comprehensive Python Notes** guide that covers fundamentals to advanced concepts.

Python Notes

Table of Contents

1. Introduction
2. Variables
3. Data Types
4. Operators
5. Type Casting
6. Input & Output
7. Strings
8. Lists
9. Tuples
10. Sets
11. Dictionaries
12. Conditional Statements
13. Loops
14. Functions
15. Lambda Functions
16. List Comprehensions
17. Modules & Packages
18. Exception Handling
19. File Handling
20. Object-Oriented Programming
21. Iterators & Generators
22. Decorators
23. Regular Expressions
24. Date & Time
25. Virtual Environment & Pip
26. Popular Libraries
27. Pandas
28. NumPy
29. Interview Questions



1. Introduction

Python is:

- High-level
- Interpreted
- Object-oriented
- Dynamically typed
- Cross-platform

```
print("Hello World")
```

2. Variables

```
name = "Ashish"  
age = 29  
salary = 85000  
  
print(name)  
print(age)
```

Multiple assignment

```
a, b, c = 10, 20, 30
```

3. Data Types

```
int  
float  
str  
bool  
list  
tuple  
set  
dict  
None
```

Example

```
x = 10  
y = 5.5  
name = "Python"  
status = True
```

Check type

```
type(x)
```

4. Operators

Arithmetic

```
+  
-  
*  
/  
%  
**  
//
```

Comparison

```
==  
!=  
>  
<  
>=  
<=
```

Logical

```
and  
or  
not
```

Membership

```
in  
not in
```

Identity

```
is  
is not
```

5. Type Casting

```
int("100")  
float("10.5")  
str(100)  
list("ABC")
```

6. Input & Output

```
name = input("Enter name:")  
print(name)
```

Formatted string

```
salary = 50000  
print(f"Salary is {salary}")
```

7. Strings

```
name = "Python"
```

Operations

```
len(name)  
  
name.upper()  
  
name.lower()  
  
name.title()  
  
name.replace()  
  
name.split()  
  
name.find()  
  
name.count()
```

Slicing

```
name[0]  
  
name[-1]  
  
name[0:4]  
  
name[::-1]
```

8. Lists

```
numbers = [10,20,30]
```

Operations

```
append()  
  
extend()
```

```
insert()  
remove()  
pop()  
clear()  
sort()  
reverse()  
copy()
```

Example

```
numbers.append(40)
```

9. Tuples

```
t = (1,2,3)
```

Immutable

```
t[0]
```

10. Sets

```
s = {1,2,3}
```

Operations

```
add()  
remove()
```

```
discard()

union()

intersection()

difference()
```

11. Dictionary

```
emp = {
    "Name": "Ashish",
    "Salary": 90000
}
```

Operations

```
emp.keys()

emp.values()

emp.items()

emp.get("Salary")

emp.update()

emp.pop()
```

12. If Else

```
if salary > 50000:
    print("High")

elif salary == 50000:
    print("Medium")

else:
    print("Low")
```

13. Loops

For Loop

```
for i in range(5):  
    print(i)
```

While

```
i = 1  
  
while i <=5:  
    print(i)  
    i += 1
```

Loop Control

```
break  
  
continue  
  
pass
```

14. Functions

```
def add(a,b):  
    return a+b  
  
print(add(10,20))
```

Default parameter

```
def greet(name="Guest"):  
    print(name)
```

15. Lambda

```
square = lambda x:x*x  
  
square(5)
```

Sort

```
students.sort(key=lambda x:x["salary"])
```

16. List Comprehension

```
square = [x*x for x in range(10)]
```

With condition

```
even = [x for x in range(20) if x%2==0]
```

17. Modules

```
import math  
  
math.sqrt(25)
```

Custom Module

```
import mymodule
```

18. Exception Handling

```
try:
    x = 10/0

except ZeroDivisionError:
    print("Error")

finally:
    print("Done")
```

19. File Handling

Read

```
f = open("data.txt")

print(f.read())

f.close()
```

Write

```
f = open("data.txt", "w")

f.write("Hello")

f.close()
```

Using

```
with open("data.txt") as f:
    print(f.read())
```

20. OOP

Class

```
class Employee:

    def __init__(self,name):
        self.name = name

    def display(self):
        print(self.name)
```

Object

```
e = Employee("Ashish")

e.display()
```

Concepts

- Class
- Object
- Constructor
- Inheritance
- Encapsulation
- Polymorphism
- Abstraction

21. Iterators

```
nums = iter([1,2,3])

next(nums)
```

Generator

```
def gen():

    for i in range(5):
        yield i
```

22. Decorators

```
def decorator(func):  
  
    def wrapper():  
        print("Before")  
  
        func()  
  
        print("After")  
  
    return wrapper
```

23. Regex

```
import re  
  
re.findall()  
  
re.search()  
  
re.match()  
  
re.sub()
```

Example

```
re.findall(r"\d+", "Age 25")
```

24. Date

```
from datetime import datetime  
  
datetime.now()  
  
datetime.today()
```

25. Pip

Install package

```
pip install pandas
```

Check

```
pip list
```

26. Libraries

Library	Purpose
Pandas	Data Analysis
NumPy	Numerical Computing
Matplotlib	Visualization
Seaborn	Advanced Charts
Plotly	Interactive Charts
Requests	API Calls
OpenPyXL	Excel
PySpark	Big Data
SQLAlchemy	Database
Psycopg2	PostgreSQL

27. Pandas

```
import pandas as pd

df = pd.read_csv("employees.csv")
```

```
df.head()

df.info()

df.describe()

df.groupby()

df.sort_values()

df.fillna()

df.drop()

df.merge()
```

28. NumPy

```
import numpy as np

a = np.array([1,2,3])

a.mean()

a.max()

a.min()

a.sum()

a.reshape(3,1)
```

29. Top Python Interview Questions

1. What is Python?
2. Mutable vs Immutable?
3. List vs Tuple?
4. Set vs Dictionary?
5. Deep Copy vs Shallow Copy?
6. `is` vs `==`?
7. `*args` vs `**kwargs`?

8. Generator vs Iterator?
9. Lambda Function?
10. Decorator?
11. OOP Principles?
12. Exception Handling?
13. Multithreading vs Multiprocessing?
14. GIL (Global Interpreter Lock)?
15. Virtual Environment?
16. What is **pip**?
17. What are Modules and Packages?
18. List Comprehension?
19. Context Manager (**with**)?
20. Python Memory Management?



Python Learning Roadmap

Python Basics



Variables & Data Types



Strings



Lists, Tuples, Sets, Dictionaries



If-Else & Loops



Functions & Lambda



Modules & Packages



File Handling



Exception Handling



OOP



Iterators & Generators



Decorators



NumPy



Pandas



Matplotlib & Seaborn



APIs & Databases



PySpark & Data Engineering

These notes provide a strong foundation for Python development, data analysis, and technical interviews.

